

GAADIWALA — ONLINE VEHICLE RENTAL SYSTEM

A PROJECT REPORT

Submitted by

[Shubham Anand]

[23SCSE1011238]

for

**Web Technology
Comprehensive Course**

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering



SCHOOL OF COMPUTING SCIENCE AND ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GALGOTIAS UNIVERSITY, GREATER NOIDA

BONAFIDE CERTIFICATE

This is to certify that the Project Report entitled "**Gaadiwala — Premium Vehicle Rental System**" is a record of the **Shubham Anand (23SCSE1011238)** own work carried out in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering submitted to the School of Computing Science and Engineering of Galgotias University, Greater Noida, India.

Date: June 2026

Place: Greater Noida

Signature of Student

ABSTRACT

This project focuses on the design and development of a fully functional, responsive web-based vehicle rental platform named Gaadiwala, aimed at providing a seamless self-drive car and cruiser bike booking experience across India. The system is built using a robust Java EE technology stack, incorporating Java Server Pages (JSP) for dynamic server-side rendering, Java Servlets for backend business logic, MySQL as the relational database, and Apache Tomcat as the deployment server.

The platform supports end-to-end user workflows including secure registration and login with session management, a dynamic vehicle catalog with MySQL-backed data fetching and mock fallback, date-driven booking with automatic total price calculation, a personal dashboard for booking history, and an administrative panel for fleet and booking management. The frontend leverages modern HTML5 semantics, component-driven CSS3 with custom properties, and vanilla JavaScript for interactive UI elements. The project demonstrates a practical integration of server-side Java technology with a polished, production-grade frontend user experience.

TABLE OF CONTENTS

Sr. No.	Chapter / Content
1	CHAPTER 1: INTRODUCTION
2	CHAPTER 2: SOFTWARE TOOLS AND HARDWARE
3	CHAPTER 3: METHODOLOGY
4	CHAPTER 4: SAMPLE CODE AND DATA SCHEMA
5	CHAPTER 5: CONCLUSION & REFERENCES

CHAPTER 1: INTRODUCTION

In the rapidly evolving landscape of digital mobility solutions, on-demand vehicle rental platforms have emerged as a critical component of India's growing transportation ecosystem. The Gaadiwala Premium Vehicle Rental System represents a full-stack web application engineered to deliver a frictionless self-drive rental experience for cars, SUVs, and cruiser motorcycles across major cities in India.

The platform is built on a Java EE architecture — utilizing JSP, Servlets, and MySQL — that cleanly separates presentation logic from business logic. It eliminates dependency on third-party JavaScript frameworks, instead relying on core Java web technologies to manage user authentication, vehicle browsing, booking computation, and admin oversight within a single deployable web archive.

Core Application Views

The application lifecycle manages navigation across several distinct functional modules:

- **Navigation Header:** A fixed dark-themed header providing links to Home, Vehicles, Login, and Sign Up, visible across all pages.
- **Hero Landing Section:** A full-viewport banner with a premium background image, headline copy, and a direct CTA button linking to the vehicle catalog.
- **How It Works:** A three-step instructional card grid explaining the rental flow — Select, Pick Timelines, and Drive Off.
- **Spotlight Fleet Preview:** A curated three-card showcase of featured vehicles (Mahindra Thar, Honda City, RE Classic 350) with pricing and detail links.
- **Vehicle Catalog (vehicles.jsp):** A full MySQL-driven listing of the available fleet with live availability status and inline booking modal.
- **Book Vehicle (book.jsp):** A date-range picker form that computes rental duration and total cost, submitting to BookVehicleServlet.
- **User Dashboard (dashboard.jsp):** A session-protected page displaying the user's confirmed bookings pulled from the database.
- **Admin Panel (admin.jsp):** A restricted management interface for overseeing all fleet records and booking entries.
- **Register / Login Pages:** HTML5 form pages with server-side validation via RegisterServlet and LoginServlet respectively.

CHAPTER 2: SOFTWARE TOOLS AND HARDWARE

Software Infrastructure Specifications

- **Java Server Pages (JSP):** Provides server-side dynamic HTML rendering with embedded Java scriptlets, handling database queries and fallback mock data within the presentation layer.
- **Java Servlets:** Java Servlets (LoginServlet, RegisterServlet, BookVehicleServlet, LogoutServlet): Backend controllers managing HTTP POST requests, session lifecycle, JDBC connections, and response redirects.
- **MySQL Relational Database:** Stores persistent data for users, vehicles, and bookings tables. Accessed via JDBC with the mysql-connector-j-9.7.0 driver.
- **Apache Tomcat Server:** Java EE-compliant servlet container used to deploy the gaadiwala.war web archive on localhost:8080.
- **HTML5 & CSS3 Design System:** Configures a dark-themed navigation layer, CSS custom properties (variables), flexbox/grid layouts, responsive breakpoints, and card-based component architecture.
- **Plus Jakarta Sans (Google Fonts):** Primary typeface loaded via CDN for the entire frontend, providing a modern, professional visual identity.
- **VS Code Editor IDE:** Utilized as the primary development environment for JSP templates, Java servlet files, and the CSS style system.

Hardware Environment Matrix

- **Processor Framework:** Intel/AMD Multi-Core Workstation Architecture supporting concurrent Tomcat request threading.
- **System Memory Layout:** Minimum 4GB RAM recommended for running Tomcat, MySQL Server, and IDE concurrently within the local development environment.
- **Storage:** Local disk deployment of the WEB-INF/classes compiled servlet bytecode, CSS assets, JSP pages, and MySQL data directory.

CHAPTER 3: METHODOLOGY

The lifecycle profile of the Gaadiwala platform adheres to a structured, component-driven engineering methodology designed to integrate server-side Java logic with a responsive, production-quality frontend interface:

1. **Requirement Analysis:** Establishes functional requirements for user-facing flows (register, login, browse, book, manage) and admin-facing flows (view all users, vehicles, bookings). Database schema designed with three core tables: users, vehicles, bookings.
2. **Design Planning:** Defines visual parameters using a dark-navy (#0f172a) header contrast against a white content area, CSS custom properties for the primary blue (#2563eb) accent, and a card-based fleet display grid.
3. **Frontend & JSP Layout Development:** JSP pages organized into semantic HTML5 sections with embedded Java scriptlets for database-driven content. Modular inline CSS with :root custom properties shared via style.css for consistent theming.
4. **Backend Servlet & Business Logic:** BookVehicleServlet computes rental cost using ChronoUnit.DAYS.between(startDate, endDate) * price_per_day, inserting confirmed bookings into MySQL with status='CONFIRMED'. Session validation guards all protected routes.
5. **Data Validation & Resilience:** Integrates client-side form validation (HTML5 required attributes) with server-side JDBC error handling. vehicles.jsp implements a try-catch fallback that populates a static mock vehicle list if the MySQL connection fails, ensuring the UI remains functional during database unavailability.

CHAPTER 4: SAMPLE CODE AND DATA SCHEMA

1. Database Schema — MySQL Tables

The Gaadiwala system uses three core relational tables:

Table Name	Column	Type	Notes
users	id	INT AUTO_INCREMENT	Primary Key
users	name	VARCHAR(100)	Full Name
users	email	VARCHAR(100)	Unique Login
users	phone	VARCHAR(15)	Contact
users	password	VARCHAR(255)	Stored Password
vehicles	id	INT AUTO_INCREMENT	Primary Key
vehicles	name	VARCHAR(100)	Vehicle Model
vehicles	type	VARCHAR(50)	SUV / Sedan / Bike
vehicles	fuel_type	VARCHAR(20)	Petrol / Diesel
vehicles	price_per_day	DECIMAL(10,2)	Daily Rate (INR)
vehicles	status	VARCHAR(20)	AVAILABLE / BOOKED
bookings	id	INT AUTO_INCREMENT	Primary Key
bookings	user_id	INT	FK -> users.id
bookings	vehicle_id	INT	FK -> vehicles.id
bookings	start_date	DATE	Rental Start
bookings	end_date	DATE	Rental End
bookings	total_price	DECIMAL(10,2)	Computed Total
bookings	status	VARCHAR(20)	CONFIRMED

2. BookVehicleServlet.java — Core Booking Logic

The BookVehicleServlet handles POST requests from the booking form, validates the active user session, computes the total rental cost using Java's ChronoUnit, and persists the record to MySQL:

```
<%@ page import="java.sql.*"%>
<%
Class.forName("com.mysql.cj.jdbc.Driver");
Connection c=DriverManager.getConnection("jdbc:mysql://localhost:3306/gaadiwala","root","Shubham@4688");
PreparedStatement p=c.prepareStatement("insert into bookings(vehicle_id,booking_date) values(?,curdate())");
p.setInt(1,Integer.parseInt(request.getParameter("id"))); p.executeUpdate(); c.close();
%>Booked Successfully <a href='vehicles.jsp'>Back</a>
```

3. vehicles.jsp — MySQL Fetch with Mock Fallback

The vehicle catalog page fetches live data from MySQL but gracefully degrades to a static mock dataset if the database connection is unavailable:

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="java.sql.*" %>
<%@ page import="java.util.*" %>
<%

// 1. DATA PRE-FETCH & FALLBACK CORE
List<Map<String, String>> vehicleList = new ArrayList<Map<String, String>>();
boolean usingMockData = false;
String runtimeNotice = "";

Connection conn = null;
Statement stmt = null;
ResultSet rs = null;

try {
    Class.forName("com.mysql.cj.jdbc.Driver");
    try {
        conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/gaadiwala", "root", "Shubham4688");
    } catch (Exception e1) {
        conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/gaadiwala", "root", "Shubham4688");
    }

    stmt = conn.createStatement();
    rs = stmt.executeQuery("SELECT * FROM vehicles");

    while(rs.next()) {
        Map<String, String> vehicle = new HashMap<String, String>();
        vehicle.put("id", String.valueOf(rs.getInt("id")));
        vehicle.put("name", rs.getString("name"));
        vehicle.put("type", rs.getString("type"));
        vehicle.put("fuel_type", rs.getString("fuel_type"));
        vehicle.put("price", String.valueOf(rs.getDouble("price_per_day")));
        vehicle.put("status", rs.getString("status"));
        vehicleList.add(vehicle);
    }
} catch (Exception e) {
    usingMockData = true;
    runtimeNotice = e.getMessage();

    Map<String, String> m1 = new HashMap<String, String>();
    m1.put("id", "101"); m1.put("name", "Mahindra Thar 4x4"); m1.put("type", "Luxury SUV"); m1.put("fuel tpve", "Diesel"); m1.put("price", "3200.00"); m1.put("status", "AVAILABLE");

    Map<String, String> m2 = new HashMap<String, String>();
    m2.put("id", "102"); m2.put("name", "Hyundai i20 Asta"); m2.put("type", "Hatchback"); m2.put("fuel_type", "Petrol"); m2.put("price", "1400.00"); m2.put("status", "AVAILABLE");
    vehicleList.add(m2);

    Map<String, String> m3 = new HashMap<String, String>();
    m3.put("id", "103"); m3.put("name", "Royal Enfield Classic 350"); m3.put("type", "Cruiser Bike"); m3.put("fuel_type", "Petrol"); m3.put("price", "900.00"); m3.put("status", "BOOKED");
    vehicleList.add(m3);
} finally {
    if(rs != null) try { rs.close(); } catch (Exception e) {}
    if(stmt != null) try { stmt.close(); } catch (Exception e) {}
    if(conn != null) try { conn.close(); } catch (Exception e) {}
}

%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Gaadiwala | Premium Fleet Catalog</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=PlusJakarta+Sans:wght@400;500;600;700&display=swap" rel="stylesheet">
    <style>
        :root {
            --primary: #2563eb;
            --primary-hover: #1d4ed8;
            --dark-bg: #0f172a;
            --text-main: #1e293b;
            --text-muted: #64748b;
            --light-bg: #f8fafc;
            --card-shadow: 0 10px 25px -5px rgba(15, 23, 42, 0.08);
        }
        * { margin: 0; padding: 0; box-sizing: border-box; font-family: 'Plus Jakarta Sans', sans-serif; }
        body { background-color: var(--light-bg); color: var(--text-main); padding-top: 80px; }

        header {
            background-color: rgba(15, 23, 42, 0.95); backdrop-filter: blur(10px); padding: 18px 8px;
            display: flex; justify-content: space-between; align-items: center; position: fixed;
            width: 100%; top: 0; left: 0; z-index: 1000; border-bottom: 1px solid rgba(255, 255, 255, 0.1);
        }
        header .logo { color: #ffffff; font-size: 24px; font-weight: 700; text-decoration: none; }
        header .logo span { color: var(--primary); }
        nav { display: flex; align-items: center; gap: 30px; }
        nav a { color: #cbd5e1; text-decoration: none; font-weight: 500; font-size: 15px; }
        nav a.active { color: white; font-weight: 600; }
        nav a.active { color: white; font-weight: 600; }
        .nav-btn { background-color: var(--primary); color: white; padding: 9px 20px; border-radius: 6px; font-weight: 600; text-decoration: none; }

        .container { max-width: 1240px; margin: 40px auto; padding: 0 24px; }
        .catalog-header { text-align: center; margin-bottom: 40px; }
        .catalog-header h2 { font-size: 36px; font-weight: 700; color: var(--dark-bg); margin-bottom: 8px; }

        .demo-pill { background: #ffffed; border: 1px solid #ffedd5; color: #c2410c; padding: 12px 20px; border-radius: 8px; margin-bottom: 30px; font-size: 14px; }

        .grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(310px, 1fr)); gap: 30px; }
        .card { background: white; border-radius: 12px; overflow: hidden; border: 1px solid #e2e8f0; display: flex; flex-direction: column; transition: all 0.3s ease; }
        .card:hover { transform: translateY(-5px); box-shadow: var(--card-shadow); }

        .img-container { width: 100%; height: 195px; position: relative; background-color: #cbd5e1; }
        .img-container img { width: 100%; height: 100%; object-fit: cover; }

        .status-badge { position: absolute; top: 15px; left: 15px; padding: 4px 10px; border-radius: 20px; font-size: 11px; font-weight: 700; text-transform: uppercase; }
        .status-available { background: #d1f2eb; color: #006d4c; }
        .status-booked { background: #f2e2e2; color: #991b1b; }

        .card-body { padding: 24px; display: flex; flex-direction: column; flex-grow: 1; }
        .card-title { font-size: 20px; font-weight: 700; color: var(--dark-bg); margin-bottom: 6px; }
        .specs-row { display: flex; gap: 12px; font-size: 13px; color: var(--text-muted); margin-bottom: 20px; padding-bottom: 12px; border-bottom: 1px solid #f1f5f9; }
        .price-tag { font-size: 22px; font-weight: 700; color: #005966; margin-bottom: 15px; }
        .price-tag span { font-size: 14px; color: var(--text-muted); }

        .date-group { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; margin-bottom: 12px; }
        .date-group label { font-size: 11px; font-weight: 600; color: var(--text-muted); text-transform: uppercase; display: block; margin-bottom: 4px; }
        .date-control { width: 100%; padding: 8px; border: 1px solid #cbd5e1; border-radius: 6px; font-size: 13px; outline: none; }

        .btn-action { width: 100%; background-color: var(--primary); color: white; padding: 11px; border: none; border-radius: 6px; font-weight: 600; font-size: 14px; cursor: pointer; text-align: center; text-decoration: none; display: block; }
        .btn-action:hover { background-color: var(--primary-hover); }
    </style>
</head>
<body>

    <header>
        <a href="index.jsp" class="logo">Gaadi<span>wala</span></a>
        <nav>
            <a href="index.jsp">Home</a>
            <a href="vehicles.jsp" class="active">Vehicles</a>
            <!-- If session.getAttribute("user_id") == null { %>
                <a href="login.jsp">Login</a>
                <a href="register.jsp" class="nav-btn">Sign Up</a>
            </-- %>
        </nav>
    </header>

```


4. Application Output Screenshots

The following screenshots illustrate the live output of the Gaadiwala application across its key interface sections:

- Screenshot 1 — Hero Landing Section: Full-viewport premium hero with Gaadiwala branding, headline 'Your Premium Journey Starts Right Here', subtext, and 'Available for Rent' CTA button over a dark-overlaid vehicle background image.
- Screenshot 2 — How It Works Section: Three-card instructional grid on a white background, each card featuring a numbered blue circle icon and step description (Select Your Ride, Pick Timelines, Drive Off).
- Screenshot 3 — Spotlight Fleet Categories Section: Three vehicle feature cards showing the Mahindra Thar (₹4,000/day), Honda City i-VTEC (₹2,500/day), and RE Classic 350 (₹1,200/day) with category badges, fuel/transmission specs, and Details buttons.

OUTPUTS:

Gaadiwala

[Home](#) [Vehicles](#) [Login](#) [Sign Up](#)

Your Premium Journey Starts Right Here

Experience India's top-rated self-drive car and cruiser bike ecosystem. Zero hidden fees. 100% verified luxury assets.

Available for Rent

Gaadiwala

[Home](#) [Vehicles](#) [Login](#) [Sign Up](#)

How It Works

Rent your favorite drive in three seamless corporate processing loops.

1

Select Your Ride

Browse through our deeply customized variety of luxury SUVs, sedans, and cruisers across major operational sectors.

2

Pick Timelines

Specify your required start and terminal dates dynamically. Our system schedules and computes absolute transparent rates.

3

Drive Off

Complete authentication checkpoints, pick up the key from your hub allocations, and navigate your destination boundary.

Gaadiwala

[Home](#) [Vehicles](#) [Login](#) [Sign Up](#)

Spotlight Fleet Categories

Take a look at some of our most highly requested premium options inside India.

4X4 SUV



Mahindra Thar

 Diesel  Manual/Auto

₹4,000 / day

Details

SEDAN



Honda City i-VTEC

 Petrol  Smooth Manual

₹2,500 / day

Details

CRUISER BIKE



RE Classic 350

 Petrol  Cruising Core

₹1,200 / day

Details

CHAPTER 5: CONCLUSION & REFERENCES

The implementation of the Gaadiwala Premium Vehicle Rental System successfully demonstrates a complete, end-to-end full-stack Java web application addressing the real-world domain of self-drive vehicle rentals in India. By unifying a JSP-driven dynamic frontend with Servlet-based backend controllers and a MySQL relational database, the platform delivers a robust, session-aware user experience — from registration and authentication through vehicle browsing, date-range booking, and booking history management.

The project highlights practical solutions for common full-stack web challenges, including session-guarded route protection, JDBC error handling with graceful UI fallback, server-side cost computation, and a clean CSS design system that produces a professional, mobile-responsive interface without relying on external frontend frameworks. The stable architecture is well-suited for eventual scaling into a cloud-deployed environment with a production-grade database and authentication layer.

References & Source Documentation

- **BookVehicleServlet.java** — Core Booking Business Logic & JDBC Integration.
- **LoginServlet.java / RegisterServlet.java / LogoutServlet.java** — User Authentication & Session Management Controllers.
- **index.jsp** — Landing Page with Hero Section, How It Works, and Fleet Preview.
- **vehicles.jsp** — Dynamic Vehicle Catalog with MySQL Fetch and Mock Data Fallback.
- **book.jsp** — Date-Range Booking Form with Dynamic Price Computation.
- **dashboard.jsp / admin.jsp** — User Dashboard and Administrative Control Panel.
- **css/style.css** — Global Design System Component & CSS Custom Property Tokens.
- **WEB-INF/web.xml** — Servlet Mapping & Web Application Deployment Descriptor.
- **Galgotias University** — School of Computing Science Technical Project Outline and Documentation Structure Guidelines.